lab7 report

PB24111621 肖瀚

一、实验目的与内容

- **实验目的**：在直接映射 Cache 的基础上，掌握多路组相联 Cache 的结构设计，实现 LRU 替换算法、FIFO、伪随机替换等策略。
- **实验内容**：
  - **任务一**：将给定的直接映射 Cache（8 行，每行 4 字，写回写分配）改造为二路组相联 Cache，并实现 LRU 替换策略。
  - **任务二**：让 Cache 支持 FIFO 替换策略和随机替换，并对三种策略进行比较。
  - **任务三**：实现参数化的 N 路组相连 Cache，支持 LRU 替换策略。

二、逻辑设计

任务一和任务三类似，全部放在任务三

任务二：支持 FIFO 和随机替换策略

Cache 采用二路组相联结构，参数化设计如下：

- 组数：8 组（ `INDEX_WIDTH = 3` ）
- 块大小：4 字（ `LINE_OFFSET_WIDTH = 2` ，128 位）
- 写策略：写回（Write-Back）+ 写分配（Write-Allocate）
- 替换策略：支持 FIFO / RAND，通过参数 `REPLACEMENT_POLICY` 选择

CPU 请求地址分解为 `tag` 、 `index` 、 `word_offset` ， `index` 同时访问两路（Way0 / Way1）的 Tag RAM 和 Data RAM，并行比较两路的 `valid` 和 `tag` ，产生命中信号 `hit_way0` / `hit_way1` 和最终的 `hit` 。若命中，根据 `word_offset` 从对应的 Data RAM 行中选出 32 位数据返回；若缺失，则根据替换策略选择一路，将旧行（若脏）写回内存，并从内存读入新行。

1. 参数定义与策略选择

```
module cache #(
    parameter INDEX_WIDTH       = 3,
    parameter LINE_OFFSET_WIDTH = 2,
    parameter SPACE_OFFSET      = 2,
    parameter WAY_NUM           = 2,
    parameter REPLACEMENT_POLICY = "FIFO"   // "FIFO", "RAND"
)
```

2. FIFO 策略实现

因为只有两路，FIFO 只需要为每个组维护一个一位指针，指示下一次替换哪一路。每次发生替换时，指针翻转。

```
reg fifo_ptr [0:SET_NUM-1];   // 0: next replace way0, 1: next replace way1

// 替换路选择
always @(*) begin
    if (!valid_way0)      way_to_replace = 1'b0;
    else if (!valid_way1) way_to_replace = 1'b1;
    else begin
        if (REPLACEMENT_POLICY == "LRU")
            way_to_replace = lru_way;
        else if (REPLACEMENT_POLICY == "FIFO")
            way_to_replace = fifo_ptr[w_index];
        else // RAND
            way_to_replace = rand_way;
    end
end

// FIFO 指针更新（在重写替换时）
always @(posedge clk or negedge rstn) begin
    if (!rstn) begin
        for (i = 0; i < SET_NUM; i = i + 1)
            fifo_ptr[i] <= 1'b0;
    end else begin
        if (REPLACEMENT_POLICY == "FIFO") begin
            if (refill) begin
                fifo_ptr[w_index] <= ~fifo_ptr[w_index];
            end
        end
    end
end
```

3. 随机策略实现

使用线性反馈移位寄存器（LFSR）生成伪随机数，取最低位作为替换路选择。

```
reg [31:0] lfsr;
reg random_bit;
wire rand_way;

always @(posedge clk or negedge rstn) begin
    if (!rstn) begin
        lfsr <= 32'hDEADBEEF;
        random_bit <= 1'b0;
    end else if (REPLACEMENT_POLICY == "RAND") begin
        random_bit <= ^(lfsr & 32'h80000057);   // 多项式 x^32 + x^6 + x^4 + x^2 + x + 1
        lfsr <= {lfsr[30:0], random_bit};
    end
end
assign rand_way = lfsr[0];
```

替换路选择时，若使用随机策略，则 way_to_replace = rand_way。

任务三：参数化 N 路组相联 Cache

任务三在任务一的基础上，将原本硬编码的二路组相联结构改造为参数化的 N 路组相联 Cache（N 支持 2、4、8、16），并实现适用于任意路数的 LRU 替换策略（通过Age字段实现）。

1. N 路信号数组化

原本为 Way0 / Way1 分别定义信号，现改为使用 Verilog 打包/解包数组，所有路的信号统一管理：

```
    wire [WAY_NUM-1:0]                valid_arr, dirty_arr;
    wire [TAG_WIDTH-1:0]              r_tag_arr  [0:WAY_NUM-1];
    wire [LINE_WIDTH-1:0]             r_line_arr [0:WAY_NUM-1];
    wire [WAY_NUM-1:0]                hit_way_arr;
    reg  [WAY_NUM-1:0]                tag_we_arr, data_we_arr;
    reg  [WAY_NUM-1:0]                w_valid_arr, w_dirty_arr;
    wire hit;
    reg  [WAY_BITS-1:0]               hit_way_idx;
    reg  [WAY_BITS-1:0]               way_to_replace;      // 组合逻辑计算的替换路
    reg  [WAY_BITS-1:0]               way_to_replace_reg;  // 锁存的替换路
```

2. LRU 替换

对于 N 路组相联，为每个组的每一路维护一个 Age 字段，值越小表示越近被访问。每次命中或替换后，将目标路的 Age 设为 0，并将其他路递增。

```
localparam WAY_BITS = (WAY_NUM <= 2)  ? 1 :
                      (WAY_NUM <= 4)  ? 2 :
                      (WAY_NUM <= 8)  ? 3 : 4;

// LRU 状态更新（命中时）
integer age_i, age_j;
    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            for (age_i = 0; age_i < SET_NUM; age_i = age_i + 1)
                for (age_j = 0; age_j < WAY_NUM; age_j = age_j + 1)
                    age[age_i * WAY_NUM + age_j] <= 0;
        end else begin
            if (hit && (CS == READ || CS == WRITE)) begin
                // 命中：命中路清零，其他路 Age 加 1（饱和至 WAY_NUM-1）
                for (age_j = 0; age_j < WAY_NUM; age_j = age_j + 1) begin
                    if (hit_way_arr[age_j])
                        age[w_index * WAY_NUM + age_j] <= 0;
                    else if (age[w_index * WAY_NUM + age_j] < WAY_NUM - 1)
                        age[w_index * WAY_NUM + age_j] <=
                            age[w_index * WAY_NUM + age_j] + 1;
                end
            end else if (refill) begin
                // 填充：被替换路清零，其他路 Age 加 1（饱和）
                for (age_j = 0; age_j < WAY_NUM; age_j = age_j + 1) begin
                    if (age_j == way_to_replace_reg)
                        age[w_index * WAY_NUM + age_j] <= 0;
                    else if (age[w_index * WAY_NUM + age_j] < WAY_NUM - 1)
                        age[w_index * WAY_NUM + age_j] <=
                            age[w_index * WAY_NUM + age_j] + 1;
                end
            end
        end
    end
```

3. 替换路选择

- 优先查找 invalid 路（最小索引优先）
- 若所有路均有效，则查找 age 最大的路

```
    // ========== 替换路选择（优先无效路，否则选 Age 最大的路） ==========
     integer ri;
    reg [WAY_BITS-1:0] max_age;
    always @(*) begin
        way_to_replace = 0;
        // 优先选无效路（最小索引优先）
        for (ri = WAY_NUM - 1; ri >= 0; ri = ri - 1)
            if (!valid_arr[ri]) way_to_replace = ri;
        // 全有效时，选择 Age 最大的路（若有多个，选最小索引）
        if (&valid_arr) begin
            max_age = 0;
            for (ri = WAY_NUM - 1; ri >= 0; ri = ri - 1) begin
                if (age[w_index * WAY_NUM + ri] >= max_age) begin
                    max_age = age[w_index * WAY_NUM + ri];
                    way_to_replace = ri;
                end
            end
        end
    end
```

4. Generate 循环例化 N 路 BRAM

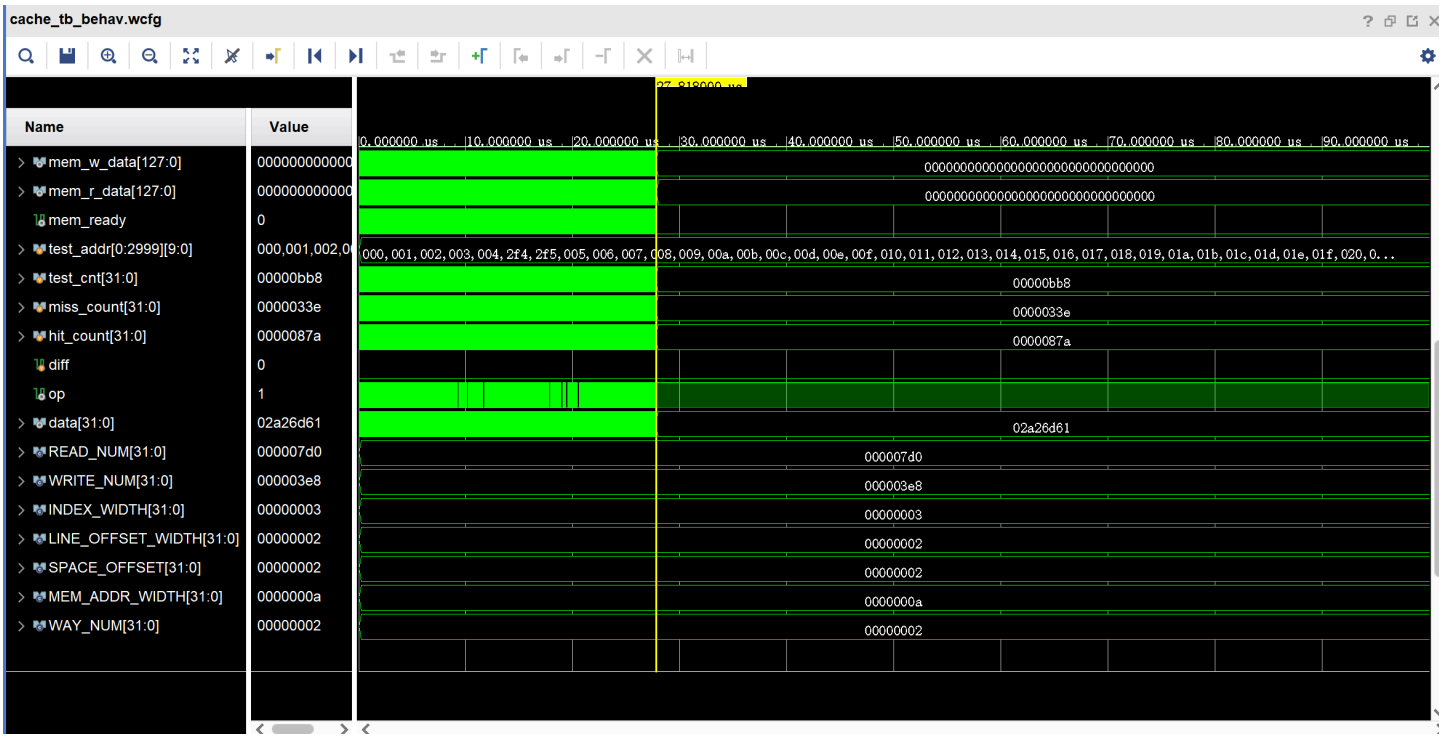
使用 `generate for` 循环替代手动实例化多路 Tag/Data RAM：

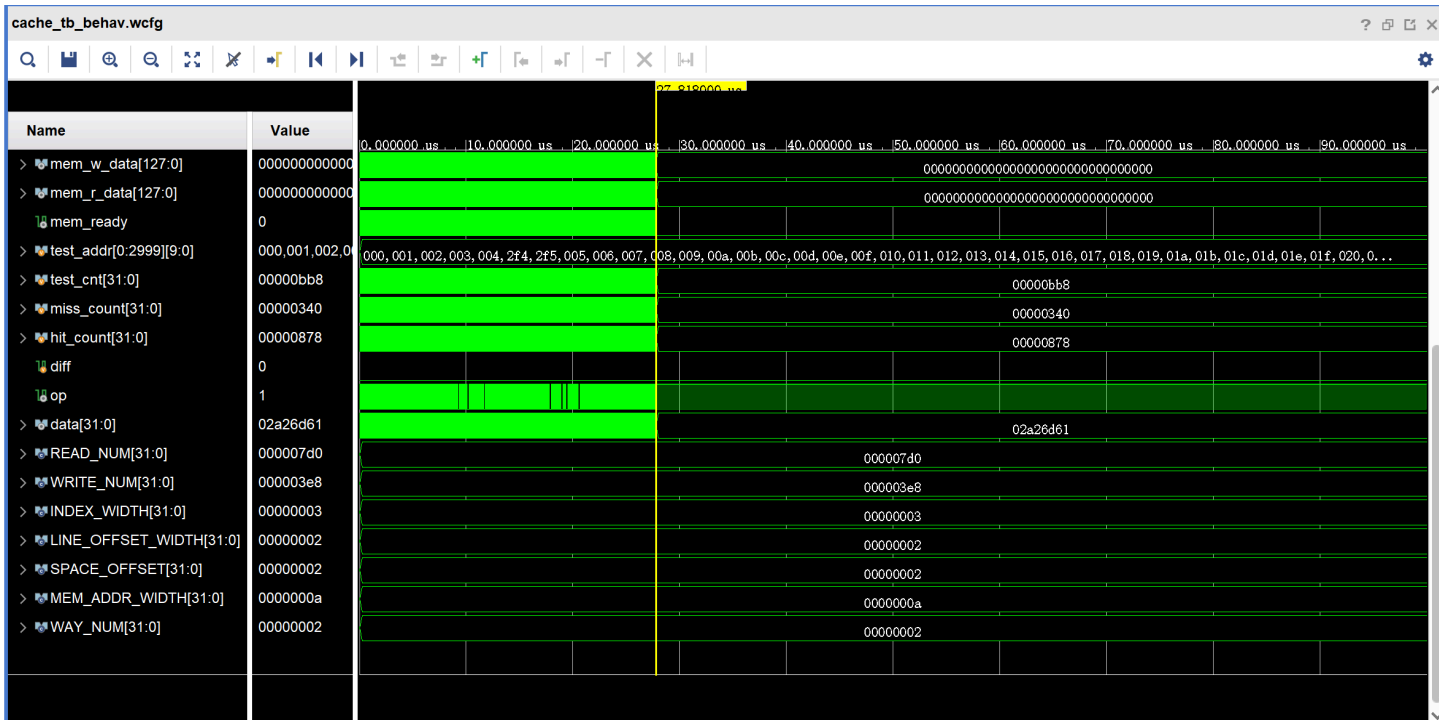
```
generate
    for (gi = 0; gi < WAY_NUM; gi = gi + 1) begin : way_block
        bram #(.ADDR_WIDTH(INDEX_WIDTH), .DATA_WIDTH(TAG_WIDTH+2))
        tag_bram (.clk(clk), .raddr(r_index), .waddr(w_index),
            .din({w_valid_arr[gi], w_dirty_arr[gi], tag}),
            .we(tag_we_arr[gi]),
            .dout({valid_arr[gi], dirty_arr[gi], r_tag_arr[gi]})
        );
        bram #(.ADDR_WIDTH(INDEX_WIDTH), .DATA_WIDTH(LINE_WIDTH))
        data_bram (.clk(clk), .raddr(r_index), .waddr(w_index),
            .din(w_line_arr[gi]), .we(data_we_arr[gi]),
            .dout(r_line_arr[gi])
        );
    end
endgenerate
```

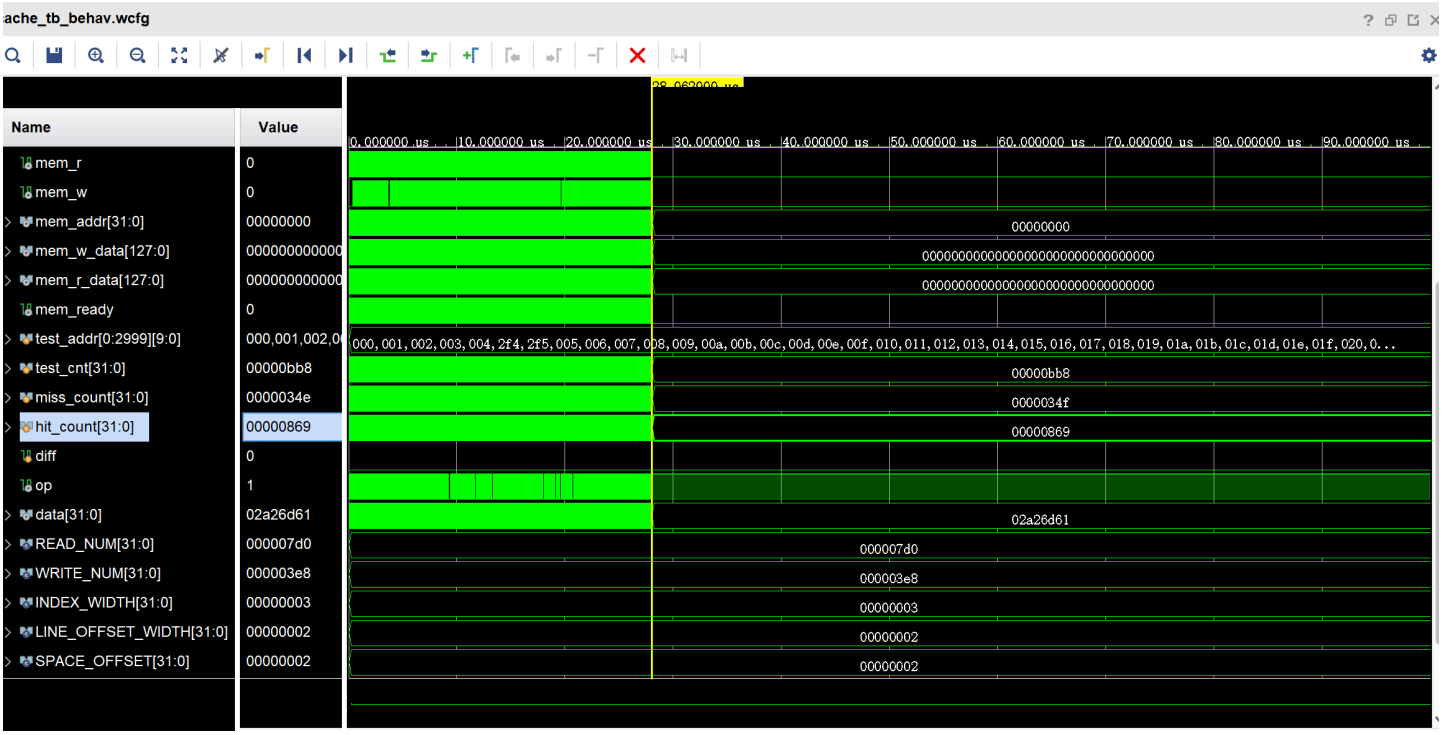
三、仿真测试

任务1和任务2的仿真

三种策略对应的仿真波形如下图：







总结得到下面的测试表格：

策略	命中	总	命中率
LRU	2170	3000	0.7233
FIFO	2167	3000	0.7223
随机	2153	3000	0.7177

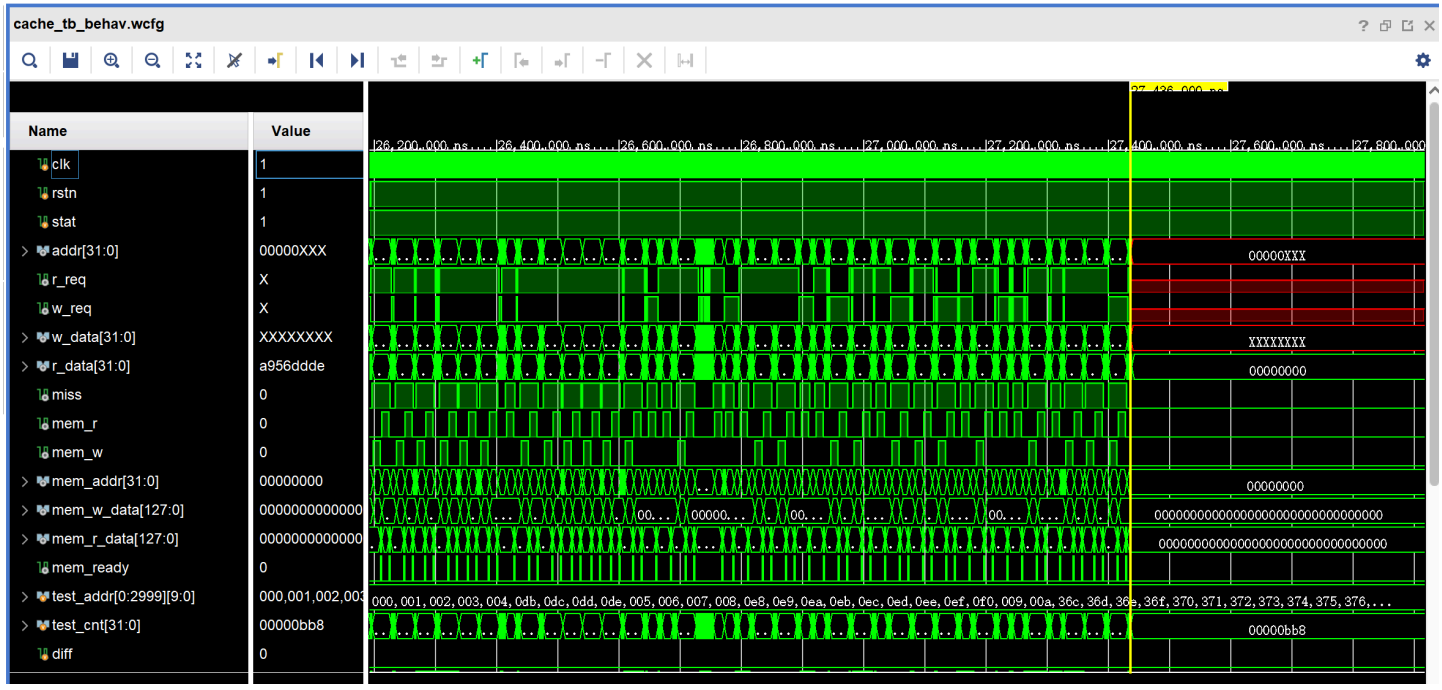
- LRU
  - 优点：命中率最高（72.33%），因为它能较好地利用程序的时间局部性，保留最近访问过的数据，淘汰最久未使用的块。
  - 缺点：实现复杂度较高（需要维护每个组内的使用顺序），但在此测试中优势明显。
- FIFO
  - 命中率略低于LRU（72.23%），差距仅为0.1个百分点。
  - 优点：实现简单（只需一个指针），但可能存在“Belady异常”(分配的缓存块增多时命中率反而下降)。
  - 缺点：对于时间局部性较强的访问模式，FIFO不如LRU精确，但总体表现接近LRU。
- 随机替换
  - 命中率最低（71.77%），比LRU低约0.56个百分点。
  - 优点：实现最简单（只需一个随机数生成器），无需记录历史信息。
  - 缺点：缺乏对访问模式的适应性，完全靠概率，因此性能最不稳定，但硬件开销最小。

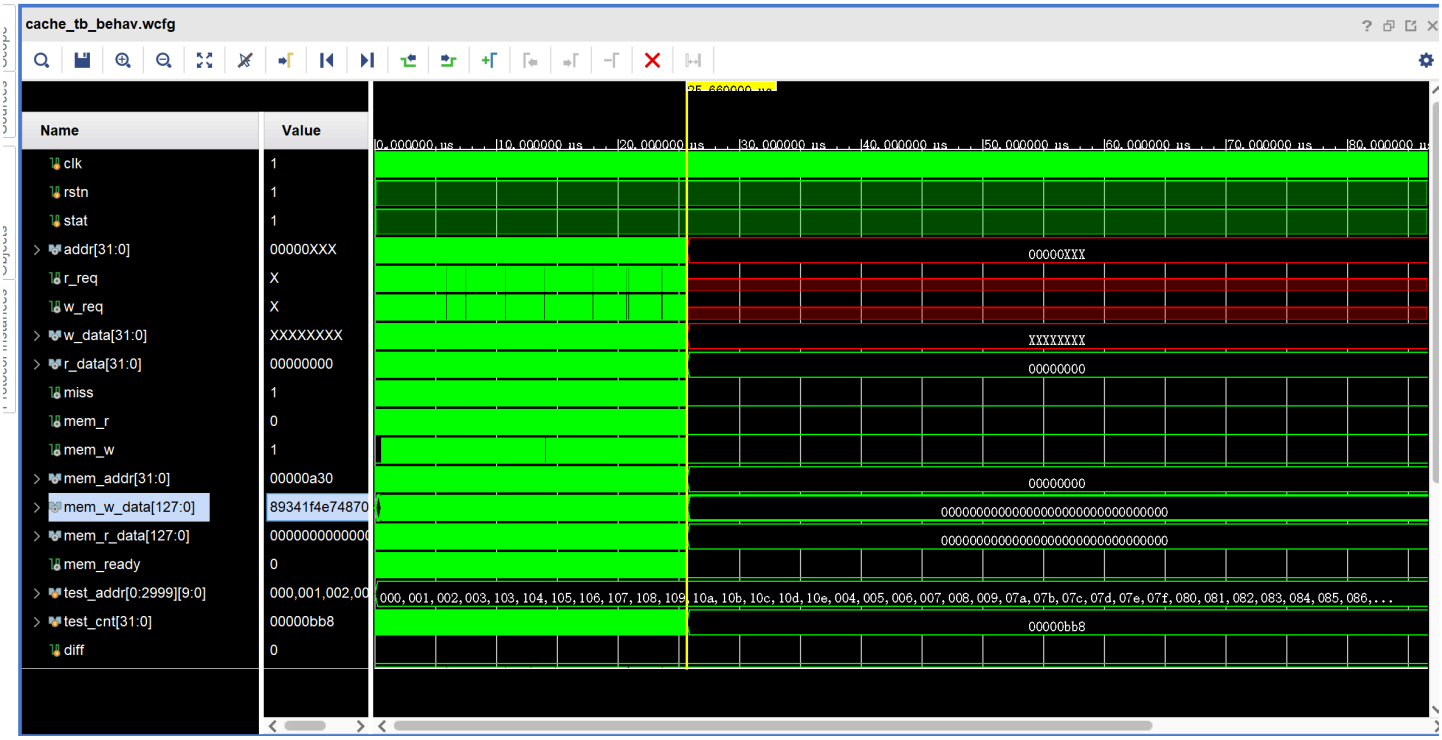
任务3的仿真

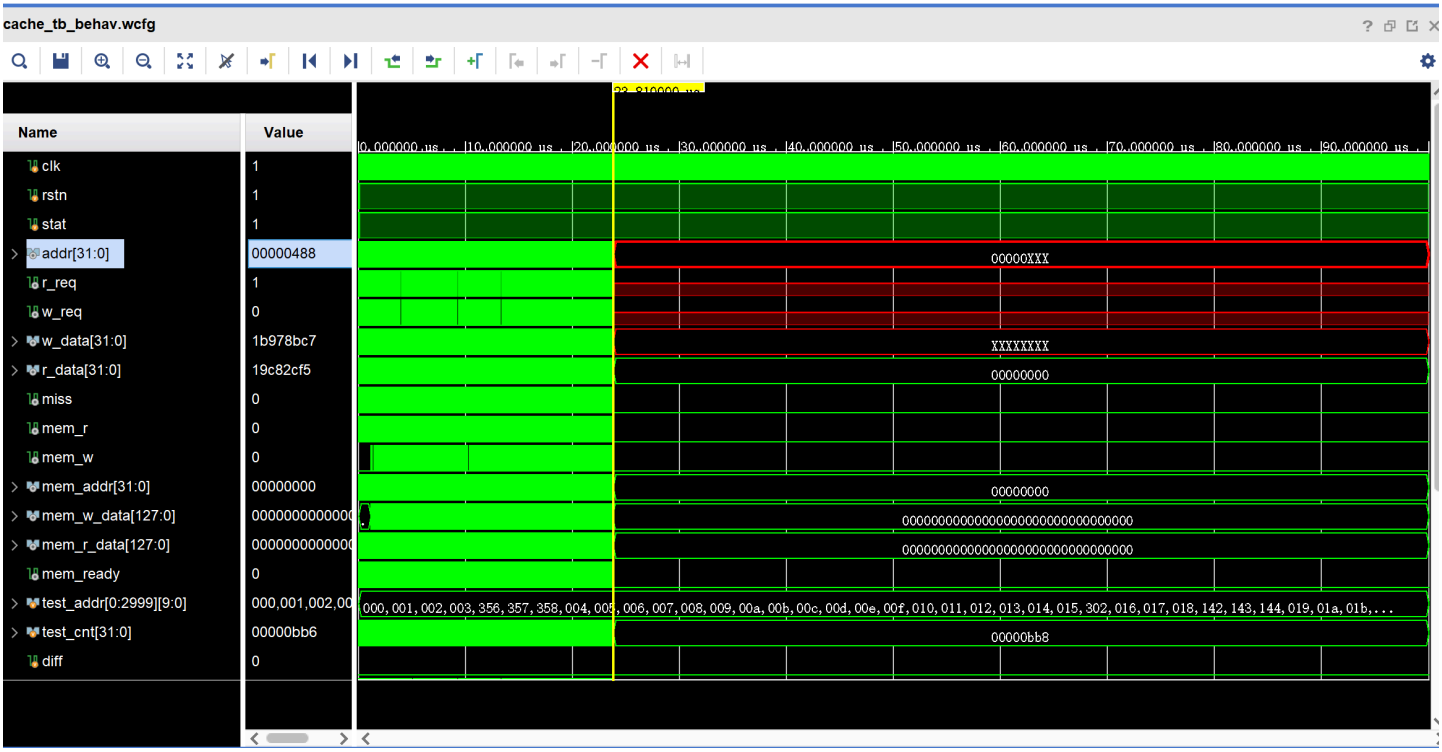
任务三的参数化设计通过改变 `WAY_NUM` 参数即可切换路数，分别配置 `WAY_NUM=2` 、 `WAY_NUM=4` 、 `WAY_NUM=8` 、 `WAY_NUM=16` 进行仿真（均采用 LRU 策略），所有配置下数据

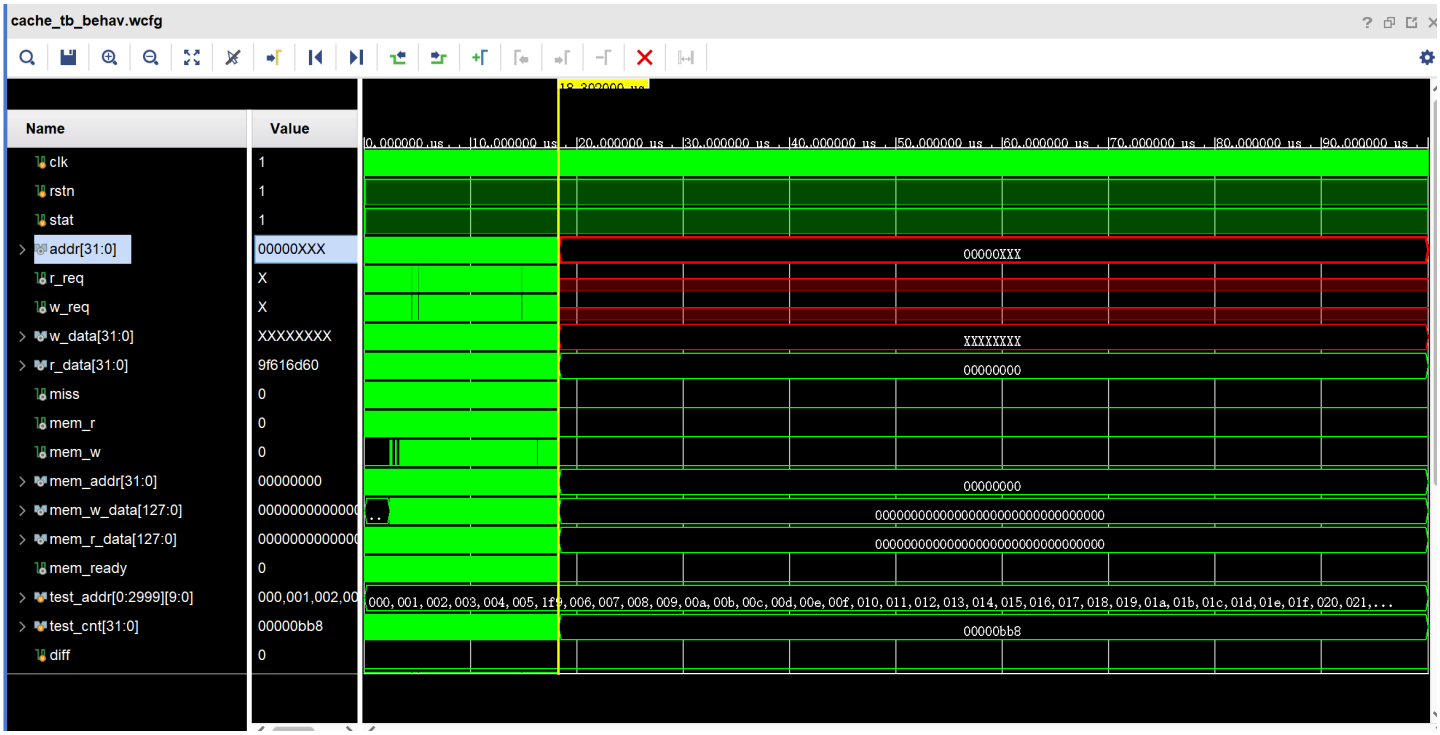
比较均无错误（`diff=0`），证明 N 路组相联 Cache 工作正常。

同时可以看到，随着相连的路数越多，测试完成时间越短，因为相联度越高，Cache 中每组能存放的内存块越多，不同地址映射到同一组的冲突概率就越低。在 LRU 策略下，这能显著降低缺失率，减少慢速内存访问次数。虽然更多路会略微增加命中时的判断延迟，但节省的内存访问时间远大于这部分开销，因此整体运行更快。









四、总结

本次实验完成了以下工作：

1. **任务一**：在直接映射 Cache 基础上实现了二路组相联结构。
2. **任务二**：扩展替换策略支持 FIFO 和伪随机替换，分别使用单 bit 指针和 LFSR 实现，同时对三种策略进行比较。
3. **任务三**：将二路设计参数化为通用的 N 路组相联 Cache（N = 2, 4, 8, 16），采用设立 Age 字段来实现 LRU 替换策略。

通过本次实验，加深了对高速缓存 Cache 的组织结构和工作机理的认识与理解。